

Raspberry Pi for life-dashboard

A first-timer's guide from unboxing to an always-on service

Prepared for Max

September 2026

Read this first

The dashboard runs on your Mac today, and that is fine: the Mac is plugged in most of the time and a missed 10pm push is not a big deal. This guide exists because you want to learn the Pi. Treat it as a project in its own right. Nothing in the dashboard depends on finishing it.

What you will have at the end: a small Linux computer next to your router that boots by itself after a power cut, runs the dashboard as a background service, receives the Health Auto Export pushes, polls Goodreads, drives the Pixoo, and copies its database back to the Mac every night.

The guide is written for macOS as the computer you sit at. Every command you type on the Mac is marked **Mac**; every command you type on the Pi over SSH is marked **Pi**. If a command fails, do not guess; copy the error into Claude Code and ask.

Contents

1	What to buy	3
2	Flash the operating system	3
2.1	Install Raspberry Pi Imager on the Mac	3
2.2	Write the card	3
3	First contact over SSH	4
3.1	Find the Pi	4
3.2	Log in	4
3.3	Update everything	4
3.4	Give the Pi a fixed address	4
3.5	SSH keys instead of passwords	4
4	Linux in ten commands	5
5	Install the tools	5
5.1	git and build basics	5
5.2	uv and Python 3.12	5
6	Get the project onto the Pi	6
6.1	Clone	6

6.2	Secrets and config	6
6.3	Smoke test by hand	6
7	Run it as a service	6
8	Move the data from the Mac	7
9	Nightly backup to the Mac	8
10	Keeping it healthy	8
11	When something goes wrong	9

1 What to buy

Item	Suggestion	Why
Board	Raspberry Pi 5, 4 GB	Plenty for a Python service. The 2 GB works too; the 8 GB is wasted here. A Pi 4 (4 GB) is fine if you find one cheap.
Power supply	Official Raspberry Pi 27 W USB-C (Pi 5) or 15 W (Pi 4)	Under-powered phone chargers cause random reboots and SD corruption. Buy the official one.
Case	Official case with fan, or any case with a heatsink	The Pi 5 throttles without cooling.
Storage	32 GB or 64 GB microSD, A2 rated, from SanDisk or Samsung	A2 rating matters for a database workload. Buy two; cards fail.
Card reader	USB-C microSD reader	To flash the card from the Mac.
Ethernet cable	Any Cat 5e or 6	Optional. Wired is steadier than Wi-Fi for a box that never moves.

You do not need a monitor, keyboard, or mouse. The Pi will be set up “headless” and you will only ever talk to it from the Mac.

Optional later: a small USB SSD replaces the microSD for durability. Skip it for the first build.

2 Flash the operating system

2.1 Install Raspberry Pi Imager on the Mac

Download it from <https://www.raspberrypi.com/software/> or, if you have Homebrew:

```
# Mac
brew install --cask raspberry-pi-imager
```

2.2 Write the card

Put the microSD in the reader and plug it into the Mac. Open Raspberry Pi Imager and choose:

1. **Device:** your board (Raspberry Pi 5 or 4).
2. **Operating System:** *Raspberry Pi OS (other)* → *Raspberry Pi OS Lite (64-bit)*. Lite means no desktop, which you do not need.
3. **Storage:** the microSD card. Check the size so you do not pick a backup drive.

Click **Next**, then **Edit Settings**. This step is what makes headless setup possible. Fill in:

- **General** tab
 - Hostname: **dashboard**
 - Username: **max**; a password you will remember
 - Wireless LAN: your home Wi-Fi name and password, country **US**. Fill this in even if you plan to use Ethernet; it is a fallback.
 - Locale: time zone **America/New_York**, keyboard **us**
- **Services** tab

- Enable SSH: on. Choose *Use password authentication* for now. You will switch to keys in Section 3.5.

Save, confirm, and let it write and verify. Eject the card, put it in the Pi, connect Ethernet if you have it, and plug in the power. The green light flickers for about a minute on first boot.

3 First contact over SSH

3.1 Find the Pi

Wait two minutes after power-on, then from the Mac:

```
# Mac
ping -c 3 dashboard.local
```

If that answers, you are done finding it. If it does not, open your router's admin page and look for a device named **dashboard** in the client list; note its IP address and use that in place of `dashboard.local` below.

3.2 Log in

```
# Mac
ssh max@dashboard.local
```

Type **yes** to the fingerprint question the first time, then your password. The prompt changes to `max@dashboard:~$`. You are now typing on the Pi. **exit** brings you back to the Mac.

3.3 Update everything

```
# Pi
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

`sudo` means “do this as administrator”. The Pi reboots and drops your SSH session; wait a minute and log in again.

3.4 Give the Pi a fixed address

Health Auto Export and the Pixoo will both point at the Pi's IP address, so it must never change. Do this on the router, not the Pi: find the DHCP settings, locate the **dashboard** client, and choose *Reserve* or *Assign static IP*. Write the address down; this guide calls it `PI_IP`.

3.5 SSH keys instead of passwords

Keys let you log in without typing a password and are safer. On the Mac, if you have never made a key:

```
# Mac
ssh-keygen -t ed25519 -C "max mac"
```

Press Enter at every prompt. Then copy the public key to the Pi:

```
# Mac
ssh-copy-id max@dashboard.local
```

Test with `ssh max@dashboard.local`; it should not ask for a password. Optionally turn password logins off:

```
# Pi
sudo sed -i 's/^#\?PasswordAuthentication */PasswordAuthentication no/'
/etc/ssh/sshd_config
sudo systemctl restart ssh
```

4 Linux in ten commands

You will use these constantly. Everything else you can look up.

Command	Meaning
<code>pwd</code>	Where am I?
<code>ls -la</code>	List files here, including hidden ones.
<code>cd dir</code> <code>cd ..</code> <code>cd</code>	Go into <code>dir</code> ; go up one; go home.
<code>cat file</code> <code>less file</code>	Print a file; page through a long one (<code>q</code> to quit).
<code>nano file</code>	Edit a file. <code>Ctrl-O</code> saves, <code>Ctrl-X</code> quits.
<code>sudo cmd</code>	Run <code>cmd</code> as administrator.
<code>systemctl status name</code>	Is the service <code>name</code> running?
<code>journalctl -u name -f</code>	Follow the service's log live. <code>Ctrl-C</code> stops.
<code>df -h</code> <code>free -h</code>	Disk space; memory.
<code>sudo reboot</code> <code>sudo poweroff</code>	Restart; shut down cleanly before pulling power.

5 Install the tools

5.1 git and build basics

```
# Pi
sudo apt install -y git curl build-essential libsqlite3-dev
```

5.2 uv and Python 3.12

Raspberry Pi OS ships Python 3.11. The project pins 3.12, and `uv` installs it without touching the system copy.

```
# Pi
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.local/bin/env
uv python install 3.12
uv --version
```

6 Get the project onto the Pi

6.1 Clone

The repo is private, so the Pi needs a way to read it. The simplest is a fine-grained GitHub token with read access to this one repository, created at <https://github.com/settings/personal-access-tokens>. Then:

```
# Pi
cd ~
git clone https://github.com/hellomaxlee/life-dashboard.git
cd life-dashboard
git checkout main
uv sync
```

When git asks for a password, paste the token. To stop it asking every time:

```
# Pi
git config --global credential.helper store
```

The Pi tracks `main`. Development stays on `dev` on the Mac; promoting `dev` to `main` by pull request is what makes a deploy.

6.2 Secrets and config

Copy the two files git does not carry. On the Mac, from the project directory:

```
# Mac
scp .env config.toml max@dashboard.local:~/life-dashboard/
```

Then on the Pi confirm they arrived and that `.env` is readable only by you:

```
# Pi
chmod 600 ~/life-dashboard/.env
ls -la ~/life-dashboard/.env ~/life-dashboard/config.toml
```

6.3 Smoke test by hand

```
# Pi
cd ~/life-dashboard
uv run pytest -q
uv run fastapi run app/main.py --port 8080
```

From the Mac, open <http://dashboard.local:8080> in a browser. If the preview page loads, stop the server with Ctrl-C. Note `fastapi run`, not `fastapi dev`: `dev` reloads on file changes and is for the Mac.

7 Run it as a service

A systemd unit starts the dashboard at boot, restarts it if it crashes, and keeps its logs. Create the file:

```
# Pi
sudo nano /etc/systemd/system/life-dashboard.service
```

Paste this, then Ctrl-O, Enter, Ctrl-X:

```
[Unit]
Description=life-dashboard
After=network-online.target
Wants=network-online.target

[Service]
User=max
WorkingDirectory=/home/max/life-dashboard
EnvironmentFile=/home/max/life-dashboard/.env
ExecStart=/home/max/.local/bin/uv run fastapi run app/main.py --port 8080
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start it:

```
# Pi
sudo systemctl daemon-reload
sudo systemctl enable --now life-dashboard
systemctl status life-dashboard
```

You want to see *active (running)*. To watch the log live:

```
# Pi
journalctl -u life-dashboard -f
```

The scheduler inside the app owns the 6:30 and 10pm work, the Goodreads poll, and the Pixoo refresh, so there is no cron to set up for those.

8 Move the data from the Mac

Do this once, when you decide the Pi is taking over. Stop the service on both machines first so nothing writes mid-copy.

```
# Pi
sudo systemctl stop life-dashboard
```

```
# Mac, from the project directory
rsync -av --progress data/ max@dashboard.local:~/life-dashboard/data/
```

rsync copies only what changed, so running it twice is safe. Then on the Pi:

```
# Pi
cd ~/life-dashboard
```

```
uv run python -m tools.replay --since 2026-01-01
sudo systemctl start life-dashboard
```

The replay recomputes every metric from the raw archive and must report zero differences against the copied database. If it reports any, stop and ask before going further.

Finally, in Health Auto Export on the phone, change both automations' URL to the Pi, and set the Pixoo adapter host in `config.toml` if it was pointing at the Mac:

```
http://PI_IP:8080/ingest/health
```

9 Nightly backup to the Mac

The backup runs from the Mac and pulls, so the Pi never needs credentials for the Mac. On the Mac, create `~/bin/pi-backup.sh`:

```
#!/bin/sh
set -e
DEST="$HOME/life-dashboard-backups/$(date +%F)"
mkdir -p "$DEST"
ssh max@dashboard.local 'cd ~/life-dashboard && uv run python -m tools.backup --out
/tmp/dash.sqlite3'
scp -q max@dashboard.local:/tmp/dash.sqlite3 "$DEST/dash.sqlite3"
rsync -a max@dashboard.local:~/life-dashboard/data/raw/ "$DEST/raw/"
echo "backup ok $DEST"
```

`tools.backup` uses SQLite's online backup API so the copy is consistent even while the service is running. Make it executable and schedule it with a launchd agent at 3am; the exact plist is in `tools/workflows/backup.md` once that tool exists. Test restore at least once: copy a backup to the Mac's own `data/` directory, run the replay, and confirm zero differences. A backup you have never restored is a hope, not a backup.

10 Keeping it healthy

- **Power cuts.** The Pi boots by itself and systemd restarts the service. SQLite runs in WAL mode so a cut mid-write loses at most the last transaction. Test it once on purpose: pull the plug, wait, plug in, check `systemctl status life-dashboard`.
- **SD card wear.** Cards die after enough writes. The nightly backup is the real protection. Keep the second card flashed and ready; recovery is "swap card, clone, copy `.env` and `config.toml`, restore data", about twenty minutes.
- **Updates.** Once a month, update the OS; whenever `main` changes, deploy the dashboard:

```
# Pi
sudo apt update && sudo apt full-upgrade -y && sudo reboot
cd ~/life-dashboard && git pull && uv sync && sudo systemctl restart
life-dashboard
```

- **Disk.** `df -h` now and then. The raw archive grows slowly; `journalctl -vacuum-time=30d` trims old logs.

- **Temperature.** `vcgencmd measure_temp`. Under 70°C is fine; above 80 means the fan or heatsink is not doing its job.
- **Shut down cleanly.** `sudo poweroff` and wait for the green light to stop before pulling power. Yanking power on a running Pi is the single most common way to corrupt a card.

11 When something goes wrong

Symptom	Try
ping <code>dashboard.local</code> fails	Wait longer after boot. Check the router's client list. Try the IP instead of the name. Confirm the Wi-Fi password in Imager was right by re-flashing.
SSH says <i>connection refused</i>	SSH was not enabled in Imager. Re-flash with the Services tab set.
SSH says <i>host key changed</i>	You re-flashed. Run <code>ssh-keygen -R dashboard.local</code> on the Mac and try again.
Service is <i>failed</i>	<code>journalctl -u life-dashboard -n 50</code> shows the last fifty log lines. Usually a missing <code>.env</code> value or a port already in use.
Preview page loads on the Pi but the phone push never arrives	Phone and Pi on the same Wi-Fi? URL uses <code>PI_IP</code> , port 8080, path <code>/ingest/health</code> ? <code>journalctl -u life-dashboard -f</code> while you tap <i>Run now</i> on the phone.
Random reboots	Almost always the power supply. Use the official one.
Everything slow, <code>free -h</code> shows no free memory	Something is leaking. <code>sudo systemctl restart life-dashboard</code> now; report it so it gets fixed.

Appendix: the whole thing on one page

1. Flash Raspberry Pi OS Lite 64-bit with hostname `dashboard`, user `max`, Wi-Fi, SSH on.
2. `ssh max@dashboard.local`; update and reboot (Section 3.3).
3. Reserve the Pi's IP on the router.
4. `ssh-copy-id max@dashboard.local`.
5. Install git and build tools; install uv; `uv python install 3.12`.
6. Clone the repo on main; `uv sync`; `scp .env config.toml` from the Mac.
7. Create the systemd unit; `enable -now`; check `status`.
8. When ready to switch: stop the service, `rsync data/`, replay, start, repoint the phone.
9. Nightly pull backup from the Mac; restore it once to prove it.